

Notebook: Tech Challenge — Previsão de Obesidade

Descrição: Notebook completo e executável para o Tech Challenge (Fase 4). Contém: EDA, limpeza e pré-processamento, pipeline reprodutível com transformadores personalizados, treino com validação cruzada e GridSearch, avaliação, salvamento de artefatos, e trechos para geração do app Streamlit e Dockerfile.

Instruções: copie este conteúdo para um novo notebook no Google Colab / Jupyter. Execute as células em ordem. Ajuste paths se necessário.

0. Setup (instalação de dependências)

```
# Execute no Colab ou ambiente local
!pip install -q scikit-learn pandas numpy matplotlib seaborn xgboost joblib
imbalanced-learn streamlit shap
```

1. Imports e constantes

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, StratifiedKFold,
GridSearchCV, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, roc_auc_score
import joblib
import shap

RANDOM_STATE = 42
DATA_PATH = 'Obesity.csv' # ajuste se necessário
```

2. Leitura dos dados e inspeção inicial

```
# carregar
df = pd.read_csv(DATA_PATH)

# primeiras linhas
print('Shape:', df.shape)
df.head()
```

```
# info e estatísticas
df.info()

# estatísticas descritivas
df.describe(include='all').T
```

3. Limpeza inicial

```
# duplicados
print('Duplicados:', df.duplicated().sum())
df = df.drop_duplicates()

# missing values
print('Missing por coluna:\n', df.isnull().sum())

# Se houver missing: política simples (documentar no PDF)
# Exemplo: dropna() – altere para fillna() se preferir
if df.isnull().sum().sum() > 0:
    df = df.dropna()

print('Shape após limpeza:', df.shape)
```

4. Conversão e rounding conforme dicionário

```
# Colunas que representam ordinal scores (ajuste conforme dicionario)
ordinal_cols = ['FCVC', 'NCP', 'FAF', 'CH20', 'TUE']
for col in ordinal_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col],
errors='coerce').round().astype('Int64')

# Se houver colunas numéricas armazenadas como string (corrigir)
# Exemplo de Height em metros com vírgula
if 'Height' in df.columns:
```

```

df['Height'] = df['Height'].astype(float)
if 'Weight' in df.columns:
    df['Weight'] = df['Weight'].astype(float)

# criar IMC (Body Mass Index) - opcional, mas informativo
if {'Weight', 'Height'}.issubset(df.columns):
    df['IMC'] = df['Weight'] / (df['Height'] ** 2)
    df['IMC'] = df['IMC'].round(2)

# checar valores únicos para categorias
for col in df.select_dtypes(include='object').columns:
    print(col, '->', df[col].unique())

```

5. Análise Exploratória Rápida (EDA)

```

# Distribuição do target
plt.figure(figsize=(8,4))
sns.countplot(data=df, x='Obesity_level')
plt.title('Distribuição de Obesity_level')
plt.show()

# Estatísticas por classe
display(df.groupby('Obesity_level').agg({'IMC': ['mean', 'median', 'std'],
    'Age': 'count'}))

# heatmap correlação (numérica)
plt.figure(figsize=(10,8))
sns.heatmap(df.select_dtypes(include=[np.number]).corr(), annot=True, fmt='.2f')
plt.title('Correlation matrix (num)')
plt.show()

```

6. Preparação da Pipeline — transformadores personalizados

```

class DropCols(BaseEstimator, TransformerMixin):
    def __init__(self, cols_to_drop=None):
        self.cols_to_drop = cols_to_drop
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        Xc = X.copy()
        if self.cols_to_drop:
            Xc = Xc.drop(columns=self.cols_to_drop, errors='ignore')
        return Xc

```

```

class ImcCalculator(BaseEstimator, TransformerMixin):
    def __init__(self, weight_col='Weight', height_col='Height',
imc_col='IMC'):
        self.weight_col = weight_col
        self.height_col = height_col
        self.imc_col = imc_col
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        Xc = X.copy()
        if self.weight_col in Xc.columns and self.height_col in Xc.columns:
            Xc[self.imc_col] = Xc[self.weight_col] / (Xc[self.height_col] **
2)

            Xc[self.imc_col] = Xc[self.imc_col].round(2)
        return Xc

```

7. Definição de colunas: numéricas, ordinais, categóricas

```

# Ajuste conforme seu dataset (baseado no dicionario)
num_cols = ['Age', 'Height', 'Weight', 'IMC'] if 'IMC' in df.columns else
['Age', 'Height', 'Weight']
ord_cols = ['FCVC', 'NCP', 'FAF', 'CH2O', 'TUE']
cat_cols = [c for c in df.columns if df[c].dtype == 'object' and c !=
'Obesity_level']

# garantir que todas as colunas existem
num_cols = [c for c in num_cols if c in df.columns]
ord_cols = [c for c in ord_cols if c in df.columns]
cat_cols = [c for c in cat_cols if c in df.columns]

print('Num:', num_cols)
print('Ord:', ord_cols)
print('Cat:', cat_cols)

```

8. Pré-processador (ColumnTransformer)

```

num_pipe = Pipeline([('scaler', MinMaxScaler())])
ord_pipe = Pipeline([('ord', OrdinalEncoder())])
cat_pipe = Pipeline([('ohe', OneHotEncoder(handle_unknown='ignore',
sparse=False))])

preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('ord', ord_pipe, ord_cols),

```

```
    ('cat', cat_pipe, cat_cols)
], remainder='drop')
```

9. Preparar X, y e split treino/teste (com stratify)

```
X = df.drop(columns=['Obesity_level'])
y = df['Obesity_level']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
stratify=y, random_state=RANDOM_STATE)
print('Train:', X_train.shape, 'Test:', X_test.shape)
```

10. (Opcional) Balanceamento com SMOTE — aplicar somente no treino

```
from imblearn.over_sampling import SMOTE

# Exemplo: se classes estiverem desbalanceadas
print(y_train.value_counts(normalize=True))

# aplicar SMOTE apenas se necessário
apply_smote = False
if apply_smote:
    # primeiro transformar X_train usando preprocessor para arrays
    X_train_proc = preprocessor.fit_transform(X_train)
    sm = SMOTE(random_state=RANDOM_STATE)
    X_res, y_res = sm.fit_resample(X_train_proc, y_train)
    print('After SMOTE:', pd.Series(y_res).value_counts())
    # NOTA: ao usar SMOTE, a persistência do pipeline precisa salvar os
    encoders separadamente
```

11. Modelagem e GridSearch (exemplo com RandomForest e XGBoost)

```
# pipeline com preprocessor + classifier
pipe_rf = Pipeline([('preproc', preprocessor), ('clf',
RandomForestClassifier(random_state=RANDOM_STATE))])
pipe_xgb = Pipeline([('preproc', preprocessor), ('clf',
XGBClassifier(use_label_encoder=False, eval_metric='mlogloss',
random_state=RANDOM_STATE))])
```

```

param_grid_rf = {
    'clf__n_estimators':[100,200],
    'clf__max_depth':[None,10,20]
}
param_grid_xgb = {
    'clf__n_estimators':[100,200],
    'clf__max_depth':[3,6,10],
    'clf__learning_rate':[0.01,0.1]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

gs_rf = GridSearchCV(pipe_rf, param_grid_rf, cv=cv, scoring='accuracy',
n_jobs=-1)
gs_xgb = GridSearchCV(pipe_xgb, param_grid_xgb, cv=cv, scoring='accuracy',
n_jobs=-1)

print('Treinando RandomForest...')
gs_rf.fit(X_train, y_train)
print('Best RF:', gs_rf.best_score_, gs_rf.best_params_)

print('Treinando XGBoost...')
gs_xgb.fit(X_train, y_train)
print('Best XGB:', gs_xgb.best_score_, gs_xgb.best_params_)

```

12. Avaliação no conjunto de teste

```

# escolher o melhor entre os dois (exemplo: comparar best_score_)
best_model = gs_xgb.best_estimator_ if gs_xgb.best_score_ >=
gs_rf.best_score_ else gs_rf.best_estimator_

# previsões
y_pred = best_model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print('\nClassification report:\n', classification_report(y_test, y_pred))

# matriz de confusão
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion matrix')
plt.ylabel('True')
plt.xlabel('Pred')
plt.show()

# ROC AUC (multiclass - one-vs-rest)
try:

```

```

y_prob = best_model.predict_proba(X_test)
# para multiclass, usar roc_auc_score com multi_class='ovo' ou 'ovr'
auc = roc_auc_score(pd.get_dummies(y_test), y_prob, multi_class='ovr')
print('ROC AUC (ovr):', auc)
except Exception as e:
    print('ROC AUC skip:', e)

```

13. Explicabilidade: Feature importance e SHAP

```

# feature importance para modelos com feature_importances_
try:
    # extrair nomes gerados pelo preprocessor
    ohe_cols = []
    if 'cat' in preprocessor.named_transformers_:
        # obter nomes do OneHotEncoder
        ohe = preprocessor.named_transformers_['cat'].named_steps['ohe']
        cat_features = ohe.get_feature_names_out(cat_cols).tolist()
        feature_names = num_cols + ord_cols + cat_features
    else:
        feature_names = num_cols + ord_cols + cat_cols

    if hasattr(best_model.named_steps['clf'], 'feature_importances_'):
        importances = best_model.named_steps['clf'].feature_importances_
        fi = pd.Series(importances,
            index=feature_names).sort_values(ascending=False)
        plt.figure(figsize=(8,6))
        fi.head(20).plot(kind='bar')
        plt.title('Top feature importances')
        plt.show()
except Exception as e:
    print('Feature importance error:', e)

# SHAP (mais custoso) – execute localmente se possível
try:
    explainer = shap.Explainer(best_model.named_steps['clf'])
    X_test_proc = best_model.named_steps['preproc'].transform(X_test)
    shap_values = explainer(X_test_proc)
    shap.summary_plot(shap_values, features=X_test_proc,
        feature_names=feature_names)
except Exception as e:
    print('SHAP skipped:', e)

```

14. Persistir artefatos (modelo e preprocessor)

```
# salvar o pipeline completo (preproc + clf)
joblib.dump(best_model, 'model_pipeline.joblib')
print('Modelo salvo: model_pipeline.joblib')

# salvar um arquivo com a ordem das colunas de entrada para uso no Streamlit
col_order = X.columns.tolist()
pd.Series(col_order).to_csv('col_order.csv', index=False)
print('Ordem das colunas salva: col_order.csv')
```

15. Gerar app Streamlit (arquivo exemplo)

```
streamlit_app = r"""
import streamlit as st
import joblib
import pandas as pd

model = joblib.load('model_pipeline.joblib')
col_order = pd.read_csv('col_order.csv', header=None)[0].tolist()

st.title('Sistema Preditivo de Obesidade')

# Criar widgets dinamicamente com base em col_order
inputs = {}
for col in col_order:
    if col in ['Age']:
        inputs[col] = st.number_input(col, min_value=1, max_value=120,
value=30)
    elif col in ['Height']:
        inputs[col] = st.number_input(col, min_value=1.0, max_value=2.5,
value=1.70, format='%.2f')
    elif col in ['Weight']:
        inputs[col] = st.number_input(col, min_value=10, max_value=300,
value=70)
    else:
        # default: text input (melhor mapear manualmente conforme dicionario)
        inputs[col] = st.text_input(col, '')

if st.button('Prever'):
    input_df = pd.DataFrame([inputs])[col_order]
    pred = model.predict(input_df)
    proba = None
    try:
        proba = model.predict_proba(input_df)
    except:
        pass
"""
```



```

        st.success(f'Predição: {pred[0]}')
        if proba is not None:
            st.write(pd.DataFrame(proba, columns=model.classes_))
    """

with open('app_streamlit.py', 'w', encoding='utf-8') as f:
    f.write(streamlit_app)

print('Arquivo app_streamlit.py gerado.')

```

16. Arquivo requirements.txt (sugestão)

```

pandas
numpy
scikit-learn
matplotlib
seaborn
xgboost
joblib
streamlit
shap
imbalanced-learn

```

17. Dockerfile (exemplo)

```

FROM python:3.10-slim
WORKDIR /app
COPY . /app
RUN pip install -r requirements.txt
EXPOSE 8501
CMD ["streamlit", "run", "app_streamlit.py", "--server.port=8501", "--server.address=0.0.0.0"]

```

18. Notas finais e recomendações



Melhorias de robustez no pré-processamento

- Aplicar **SimpleImputer** para dados numéricos/categóricos, mesmo se não houver NaNs aparentes, garantindo resiliência.
- Incluir seção de **tratamento de outliers** (boxplots + winsorization opcional).
- Criar célula Markdown explicando **por que** usar OneHotEncoding para nominais e OrdinalEncoding para variáveis com ordem.

Melhorias na EDA

- Adicionar gráficos: distribuição de Age, Weight, Height, IMC estratificados por Obesity_level.
- Adicionar violinplots e pairplots controlados.
- Incluir tabela descritiva por classe.

Modelagem mais completa

- Adicionar testes com **SVM, KNN, Logistic Regression** e registrar todas as métricas em um dataframe comparativo.
- Incluir tabela final comparando: acurácia, precisão macro, recall macro, F1 macro.
- Adicionar plot das curvas ROC para cada classe.

Modularidade para deploy

- Criar seção recomendando mover transformadores para **utils.py**.
- Documentar claramente como o Streamlit lê `col_order.csv` e o modelo.

Storytelling e visão de negócio

- Adicionar seção Markdown explicando **importância clínica** das principais variáveis.
- Relacionar achados do modelo com recomendações para equipe médica (ex.: pacientes com hábitos alimentares X têm maior risco segundo SHAP).

para o PDF final - Documente todas as decisões (por que usar SMOTE ou não, por que escolher XGBoost, quais hiperparâmetros).

- Inclua as métricas em uma tabela comparativa para todos os modelos testados.
- Adicione screenshots do Streamlit e do dashboard (Power BI/Plotly) no PDF.
- Inclua instruções passo-a-passo de como rodar (local, Docker, deploy no Streamlit Cloud).

Próximo passo: se quiser, eu posso agora gerar o arquivo `.ipynb` pronto para download (com este conteúdo convertido para células), ou gerar diretamente o `app_streamlit.py` completo com tratamento de inputs para cada coluna do dicionário. Informe qual você prefere.